KP07 The Backpropagation Algorithm

> Level: undergraduate, year 3
>
> Pattern Recognition | Chapter 4, §4.7

Slides: read and download

Learning objectives

- Write the four steps of backpropagation training and explain epoch, batch training and the stop rule.
- Walk through the first epoch of the XOR 2-2-1 network and read the curve of the cost J against epochs.
- Explain how the learning rate, the initial values, local minima and pattern training each affect training.

KP07 The Backpropagation Algorithm

One-sentence summary: KP06 took one step on one sample; real training gives the weights start values once and then repeats forward, backward and update in every epoch until the cost is small enough. The learning rate, the start values and whether training falls into a local minimum decide how fast it is and whether it succeeds.

1. Training repeats four steps

Plain explanation: Real training has many samples, for example the four XOR points. The question is when the weights are updated.

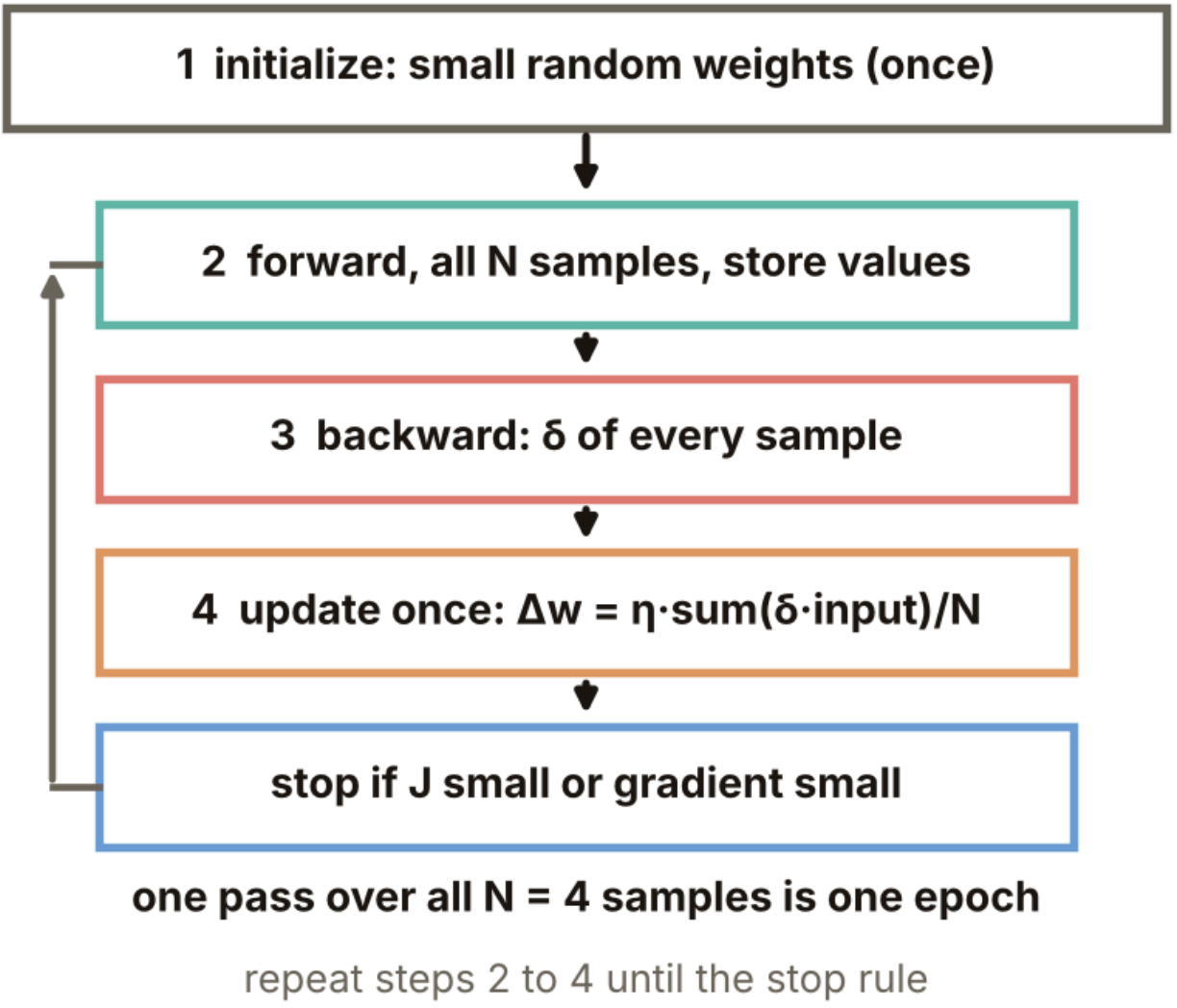
- **Epoch**: presenting all N training samples once. Here $N = 4$.
- **Batch training**: in one epoch every sample first gets its δ and the sum updates the weights once, $\Delta w = \eta \sum_i \delta(i)\, y^{r-1}(i)/N$. Dividing by N only rescales η.

Formal definition: batch update $\Delta w = \frac{\eta}{N} \sum_{i=1}^{N} \delta(i)\, y^{r-1}(i)$, where $y^{r-1}(i)$ is the previous-layer output for sample i (the input of a bias is fixed at 1).

Example: for XOR with $N = 4$, one epoch looks at the four points and updates once.

Steps: 1 give every weight a small random start value (once); 2 forward and store values; 3 backward for the δ of every sample; 4 update. Repeat 2 to 4 until stopping.

Figure `lec05-kp07-loop`: the four-step procedure; one pass over the $N = 4$ samples is one epoch.



2. Epoch 1: the four points go forward first

Take the initial weights of the source's interactive example (bias, x_1, x_2): hidden unit 1 $(-2, 5, 5)$, unit 2 $(-7, 5, 5)$, output $(-2, 8, -12)$. The outputs: $(0, 0)$ gives 0.2578 and $(1, 1)$ gives 0.0043 (target 0); $(0, 1)$ and $(1, 0)$ give 0.9851 (target 1). The cost J

is the mean of $\frac{1}{2}(\text{target} - o)^2$ over the four samples, $J = 0.0084$ (program). All four are already on the right side of 0.5.

Figure `lec05-kp07-epoch1-forward`: hidden outputs and final output of the four inputs.

input	target	h_1	h_2	output o	
(0,0)	0	0.1192	0.0009	0.2578	■
(0,1)	1	0.9526	0.1192	0.9851	●
(1,0)	1	0.9526	0.1192	0.9851	●
(1,1)	0	0.9997	0.9526	0.0043	■

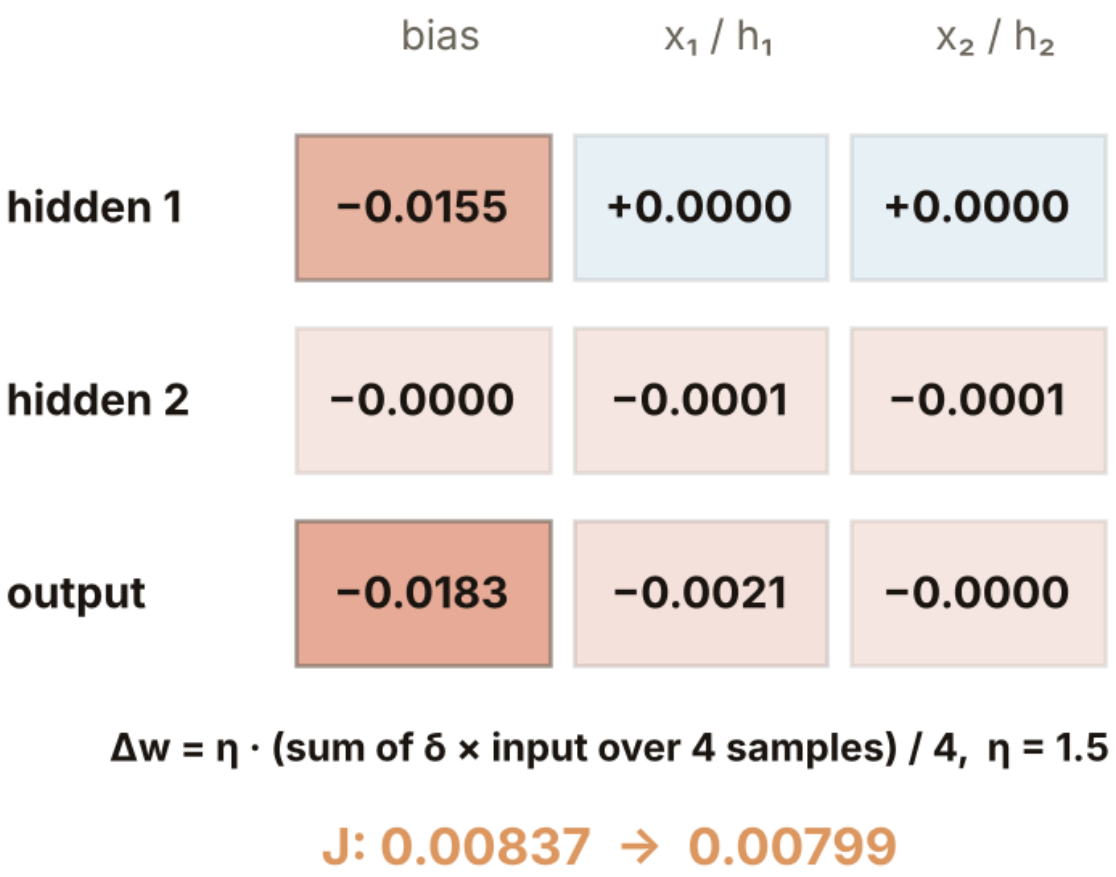
J = mean of ½(target − o)² = 0.0084

all four already on the right side of 0.5

3. Epoch 1: backward and update

The output δ_o of each sample: -0.0493 for $(0, 0)$, about $+0.0002$ each for $(0, 1)$ and $(1, 0)$, about -0.00002 for $(1, 1)$. Sum the four "$\delta\times$ input" terms, divide by 4 and multiply by $\eta = 1.5$ to update the nine weights once: output bias -0.0183, hidden unit 1 bias -0.0155, the rest smaller. After one epoch J goes from 0.00837 to 0.00799 (program), a small step.

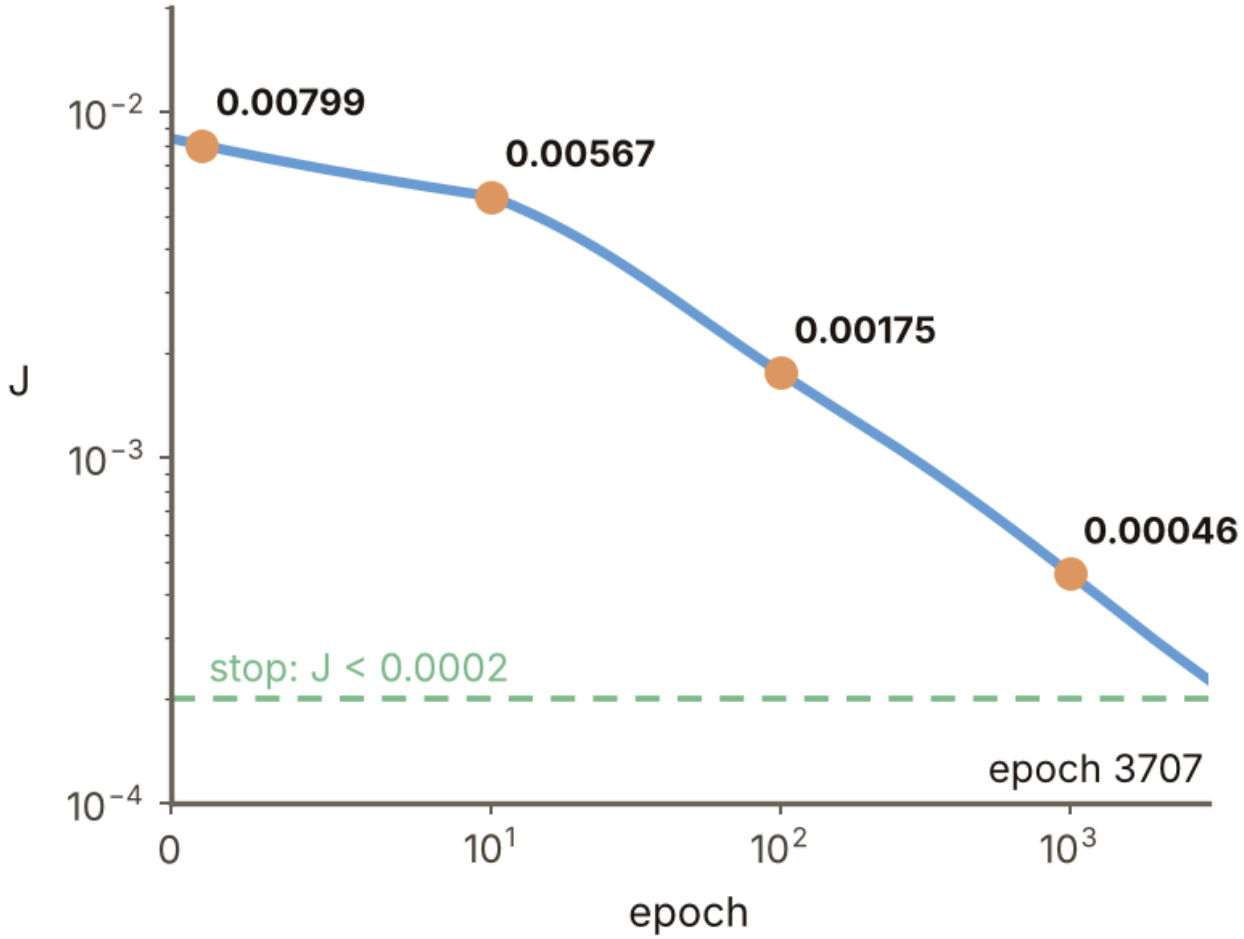
Figure `lec05-kp07-epoch1-update`: the nine weight updates, shading shows size.



4. J falls slowly to the bar

Stop rule: the cost J falls below a threshold, or the gradient becomes almost zero. Here $J < 0.0002$ and all four points correct. Repeating epochs, J is 0.00799, 0.00567, 0.00175, 0.00046 at epochs 1, 10, 100, 1000 and falls below the threshold only at epoch 3707, when the outputs are 0.029, 0.982, 0.982, 0.010 (program).

Figure `lec05-kp07-curve`: the J curve on a log axis and the stop threshold.

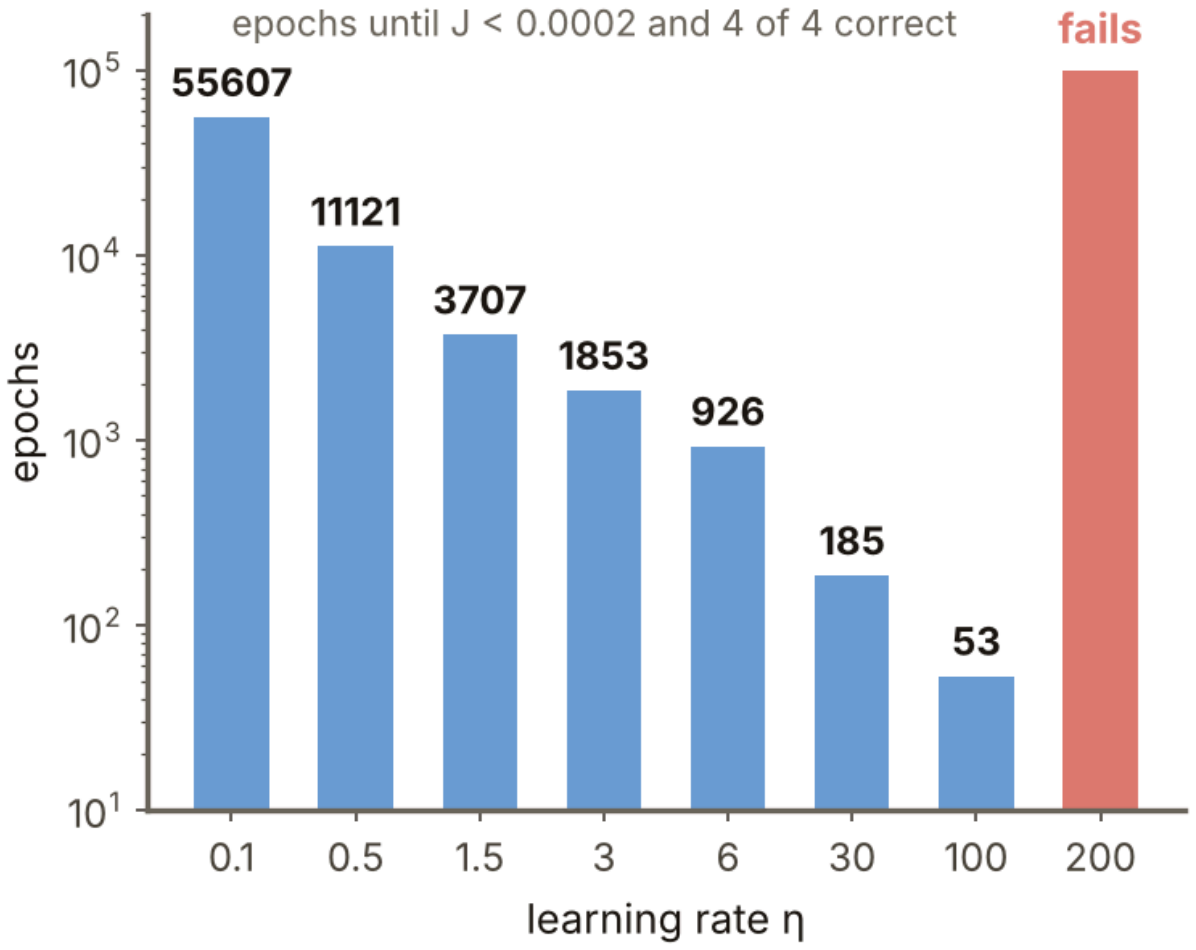


5. The learning-rate dilemma

With the source's initial weights the epochs needed against η: $0.1 \to 55607$, $0.5 \to 11121$, $1.5 \to 3707$, $3 \to 1853$, $6 \to 926$, $30 \to 185$, $100 \to 53$ (program). At $\eta = 200$ J grows in the first two steps, all four outputs end at 1.00 with $J = 0.25$ and only 2 points are correct: training fails. The source says the best η depends on the shape of the cost in weight space: flat minima allow big steps and steep narrow ones need small steps.

Common misconception: a larger learning rate is always better. Too large a step overshoots or even diverges; here a larger η is faster only within a range.

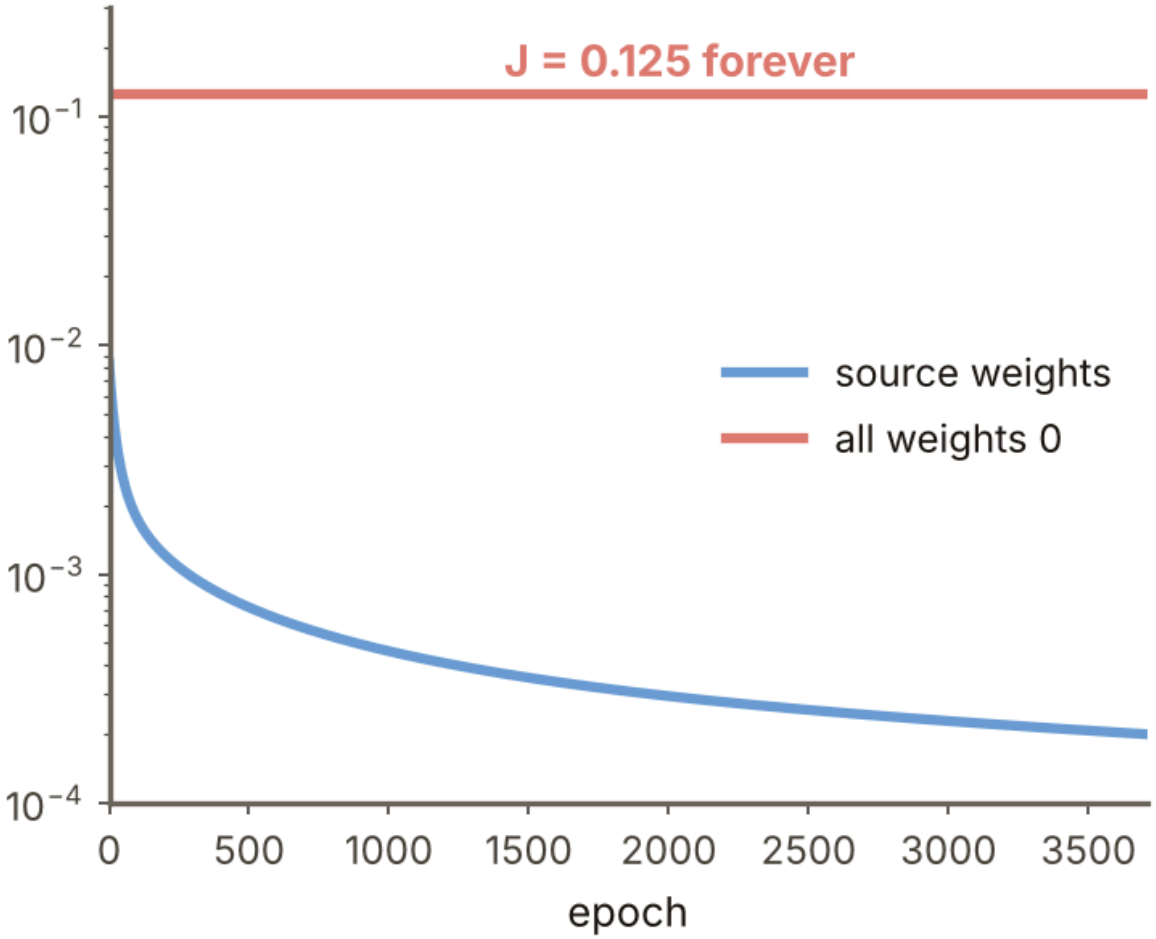
Figure `lec05-kp07-eta-bars`: epochs needed for different η, log axis.



6. The start cannot be all zeros

Initialization: before training every weight gets a start value, usually small random values (from a pseudo-random generator). With all weights 0, every output is 0.5, the output-layer δ of the four samples cancel, the hidden δ is multiplied by the output weights (0), so every gradient is exactly 0 and $J = 0.125$ never moves (program). With 20 random starts in ± 0.5 and $\eta = 1.5$ all reach $J < 0.0002$, in 10890 to 23926 epochs.

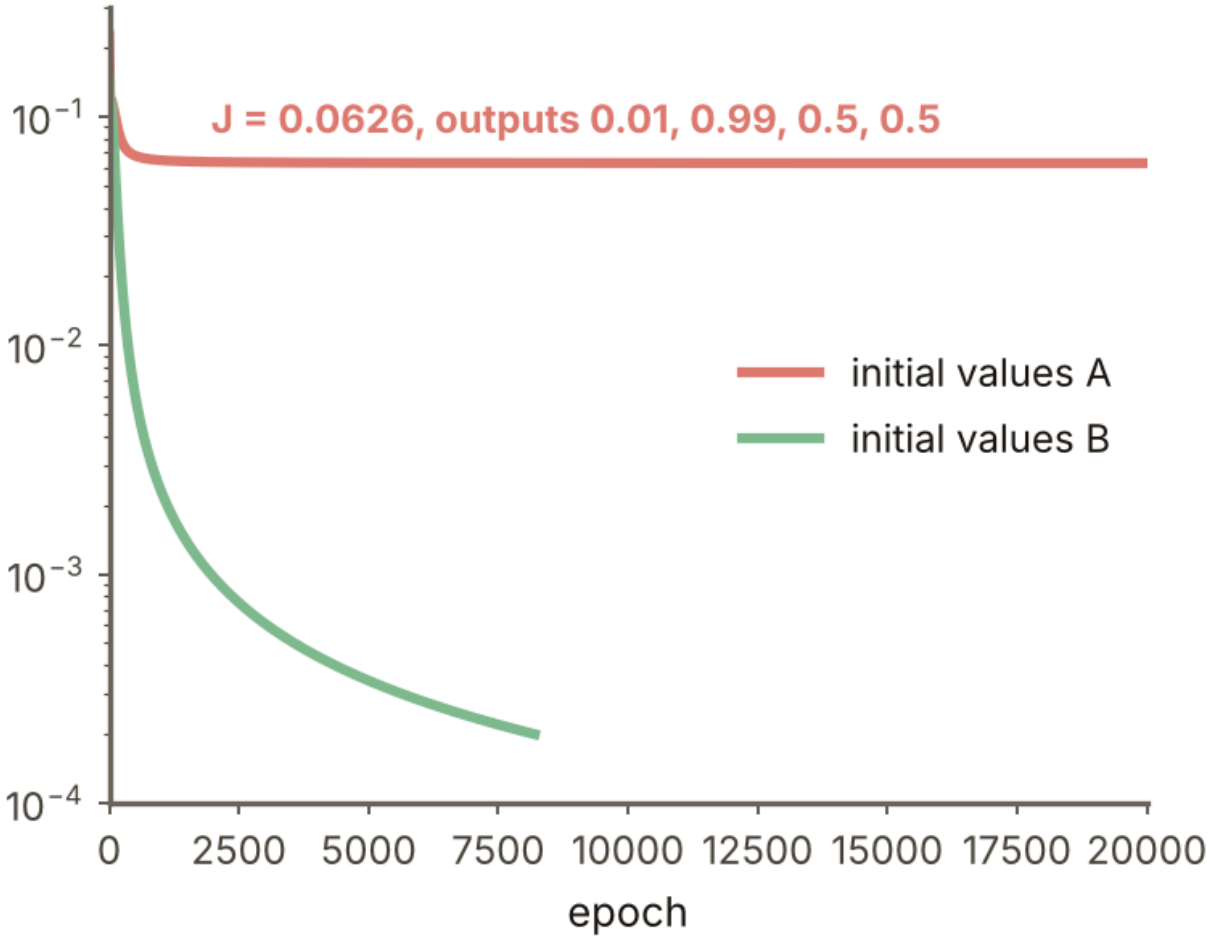
Figure `lec05-kp07-zero-init`: J curves of all-zero and source start values.



7. Local minimum

Local minimum: lowest nearby but not lowest overall, with zero gradient, so gradient descent cannot leave it. With 300 random starts in ± 3 ($\eta = 3$, at most 20000 epochs): 176 succeed and 124 do not. Start A (seed 0) stalls at $J = 0.0626$ with outputs 0.01, 0.99, 0.5, 0.5 and a gradient norm of only 3×10^{-5}; 60000 more epochs do not move it. Start B (seed 4) converges in 8204 epochs (program).

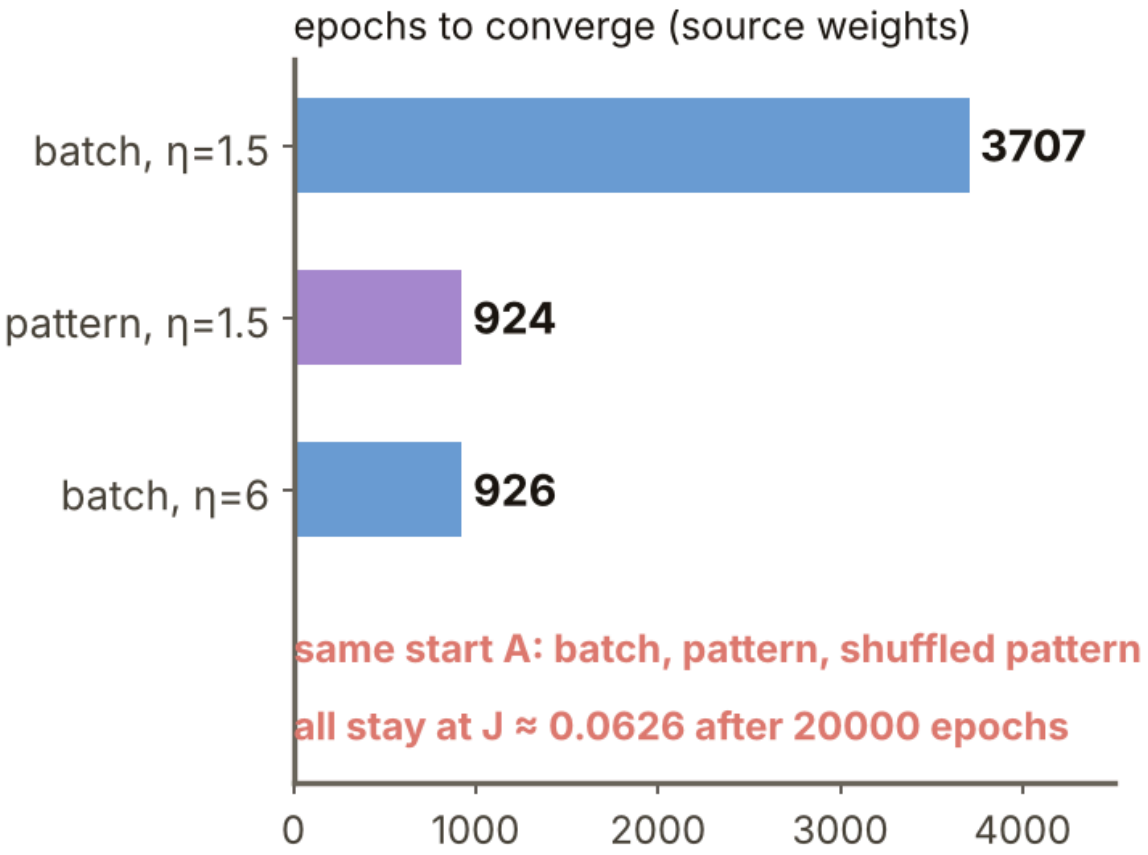
Figure `lec05-kp07-stuck`: J curves of the two starts.



8. Per-sample updates

Pattern (online) training: after every sample update with that sample's δ, $w \leftarrow w + \eta\,\delta\,y^{r-1}$; it is noisier and the source says it may jump out of a local minimum. With the source's weights: pattern $\eta = 1.5$ needs 924 epochs, batch $\eta = 1.5$ needs 3707, batch $\eta = 6$ (same total step) needs 926, so the speed gain of pattern mode is mostly step size. On stuck start A pattern mode (fixed order or shuffled each epoch) is still at $J \approx 0.063$ after 20000 epochs; it did not escape here. "May" is no guarantee (program).

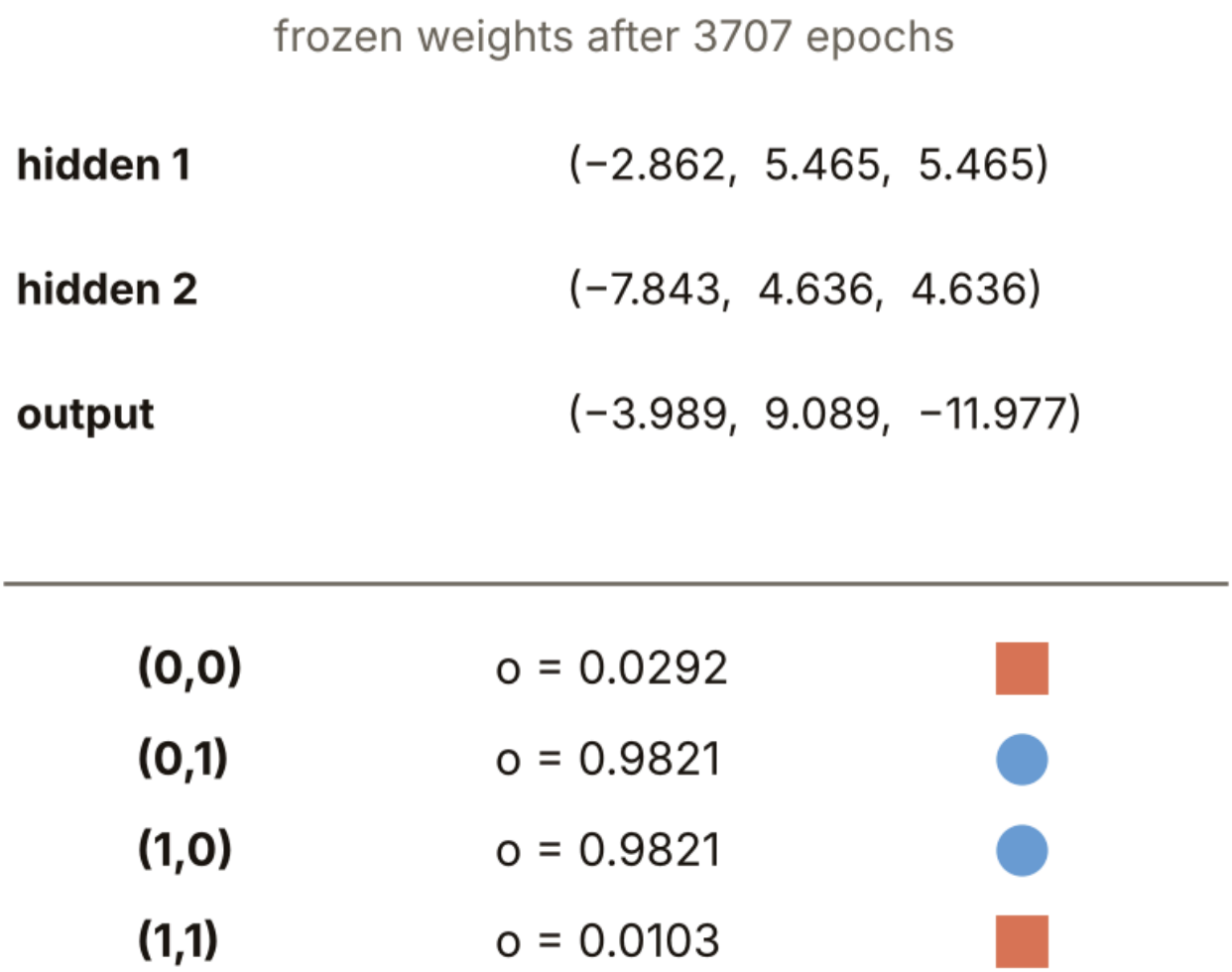
Figure `lec05-kp07-batch-pattern`: epochs of the three setups and the result from start A.



Extension: frozen weights, practice and worked solution

Frozen weights: after training the weights are no longer updated and classifying needs only multiply-adds and the sigmoid. Here the nine weights are hidden 1 $(-2.862, 5.465, 5.465)$, hidden 2 $(-7.843, 4.636, 4.636)$, output $(-3.989, 9.089, -11.977)$; each input costs 6 multiply-adds and 3 sigmoids, and the four outputs are 0.0292, 0.9821, 0.9821, 0.0103. Units of one layer are independent and can run in parallel, with optical, VLSI and dedicated neurocomputer implementations (source).

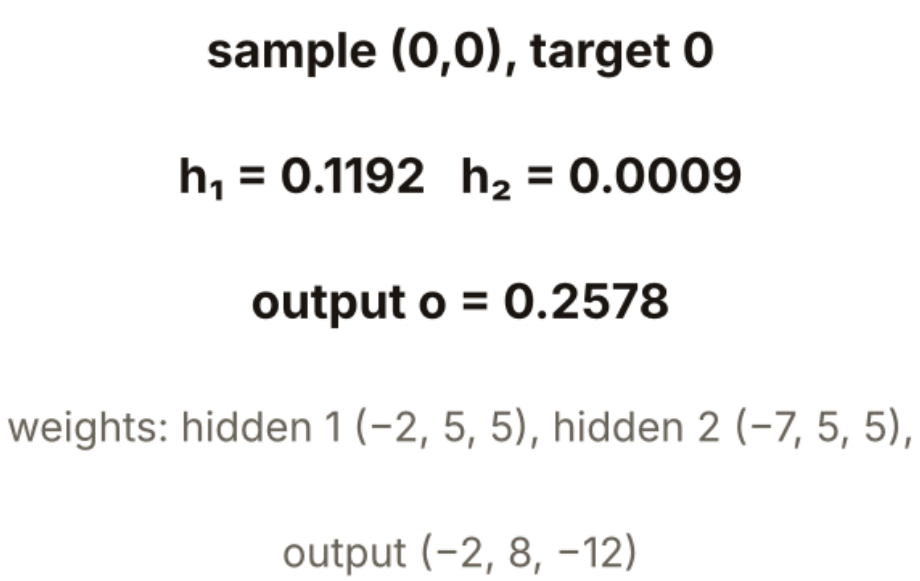
Figure `lec05-kp07-frozen`: the frozen weights and the four outputs.



Practice: with the initial weights, pattern mode on the sample $(0, 0)$ only (target 0), $\eta = 1.5$. The forward pass gives $h_1 = 0.1192$, $h_2 = 0.0009$, $o = 0.2578$.

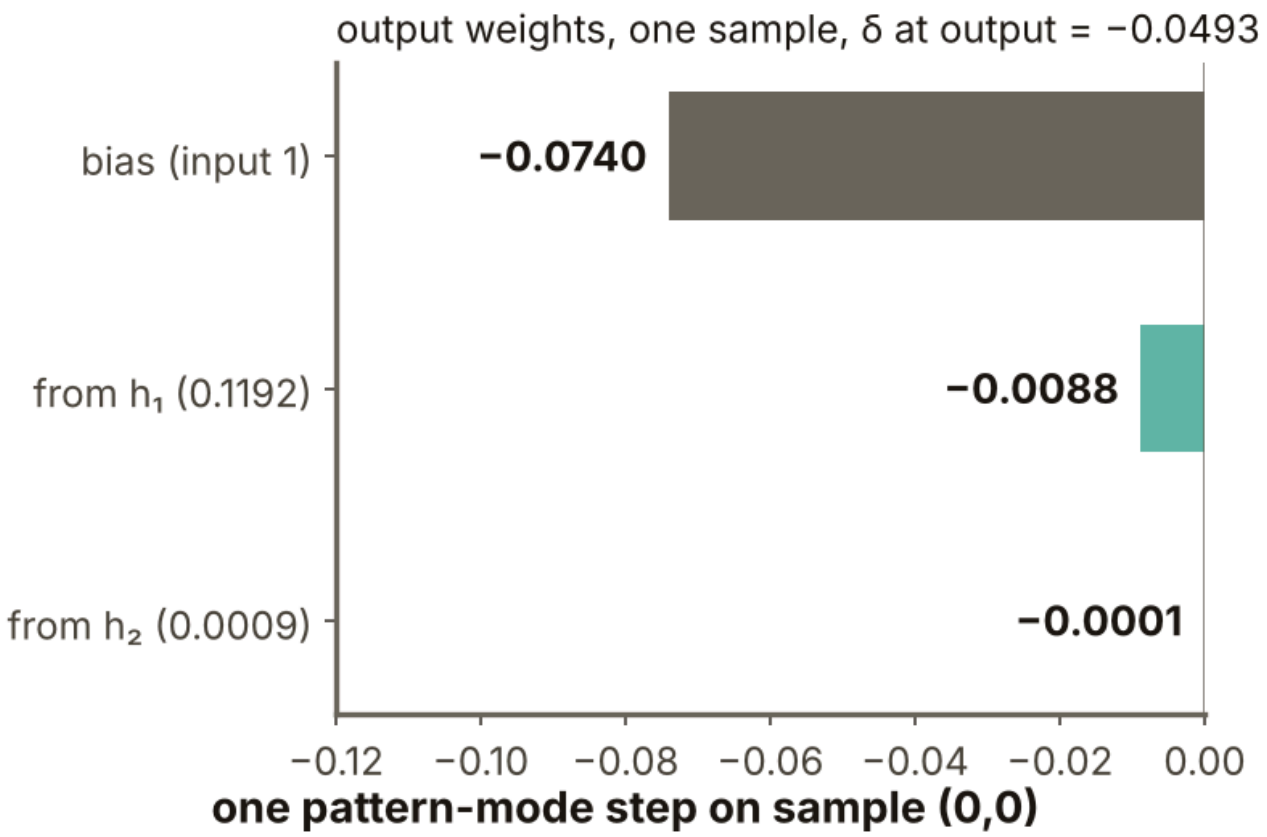
1. What is the output-layer δ_o? Hint: $\delta_o = (\text{target} - o) f'(v_o)$ and $f'(v_o) = o(1 - o)$.
2. What are the updates of the three output weights?

Figure `lec05-kp07-practice-forward`: the forward pass of the sample $(0, 0)$.



Worked solution of questions 1 and 2: (1) $\delta_o = (0 - 0.2578) \times 0.2578 \times (1 - 0.2578) = -0.0493$. (2) $\Delta w = 1.5 \times (-0.0493) \times \text{input}$: bias -0.0740, w_{o1} -0.0088, w_{o2} -0.0001. (3) Check: the target is 0 and the output 0.2578 is too high, so the output weights go down; hidden unit 1 has $\delta = -0.0414$ and its bias changes by -0.0622.

Figure `lec05-kp07-practice-sol`: the updates of the three output weights, all negative.



Terms and sources

English	中文
Epoch	epoch
Batch training	批次訓練
Stop rule	停止條件
Initialization	初始化
Local minimum	區域極小值
Pattern (online) training	逐筆訓練
Frozen weights	凍結權重

Source: Chapter 4 KP07, §4.7 (read 2026-10-25). The four steps, the batch and pattern modes, the discussion of learning rate and initial values, local minima and frozen weights follow the source; the XOR 2-2-1 network and initial weights are from the source's interactive example, and every epoch count, output and update was

recomputed for this lecture ([added], (program)). The all-zero start, the 300 random starts in ± 3 and the stuck-in-pattern-mode experiment are this lecture's own program experiments ([added]). Notation follows KP06: $\delta = -\partial E / \partial v$ and the update is $+\eta\,\delta \cdot$input.